

Design + How to compile and run code

To implement the fuzz testing for this assignment, I first had to decide which programming language would be best to use. After doing some research and consulting with peers, I determined that Python would be the easiest choice because it is simple to work with. My strategy for implementing fuzzing was to make sure the program being tested ran multiple times with different modified versions to see which bugs would be detected. I created a mutation-based fuzzer by converting the provided cross.jpg file into a byte array so that the file could be adjusted at the byte level. The byte positions were randomized to ensure that different parts of the file were modified each time the fuzzer ran. To trigger different bugs, I adjusted both the number of bytes mutated per file and the number of mutation runs in my source code. The mutation rules I used included: replacing bytes with random values, flipping bytes, swapping bytes, duplicating bytes, inserting short sequences of random bytes, and setting bytes to zero. Some mutations specifically targeted only the first 150 bytes of the file to increase the chance of getting header-related bugs. When a bug was triggered, the fuzzer saved the corresponding mutated file as test-x.jpg. By running several trials and experimenting with the number and type of mutations, I was able to discover all 10 bugs as indicated in the screenshot in the proof of bugs section of this report. However, after doing one final trial run I found 9 out of 10 bugs as bug #4 did not show, therefore it is excluded from my trigger count but I did still include it in the screenshot since the test-4.jpg was saved.

How to compile & run code

1. Log in to the UCF Eustis Linux machine.
2. Navigate to the directory where the proper files are located:
 - fuzzer.py (*my source fuzzing code submitted via canvas*)
 - jpeg2bmp (*given in assignment instructions*)
 - cross.jpg (*given in assignment instructions*)
3. Make sure the 'jpeg2bmp' program is executable by typing this command into the terminal:

chmod u+x jpeg2bmp

4. Run the fuzzer using Python 3 by typing this command into the terminal:

python3 fuzzer.py

Note: The fuzzer will create an additional directory called 'PA2_mutations' with the mutated images generated.

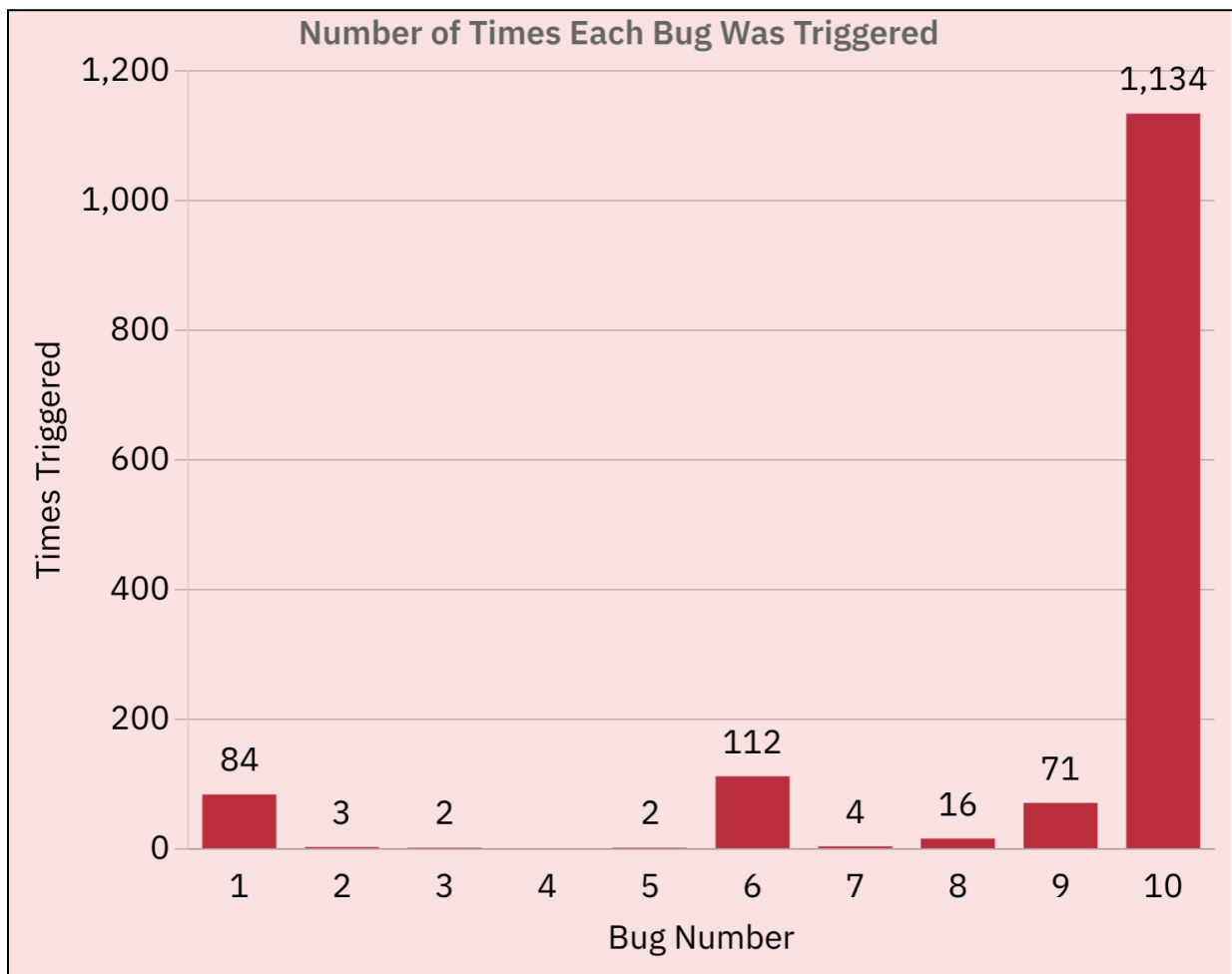
- Verify that a bug is actually triggered using this command:

`./jpeg2bmp test-x.jpg out.bmp`

Note: Replace 'x' in above command with the bug number for the specific test image.

Experimental Results

	Bug #1	Bug #2	Bug #3	Bug #4	Bug #5	Bug #6	Bug #7	Bug #8	Bug #9	Bug #10	Overall
# of crashes triggered	84	3	2	0	2	112	4	16	71	1134	1428



Proof of Finding Bugs

```

[av662490@net1547:~/CAP6135/PA2$ ./jpeg2bmp test-1.jpg out.bmp
CAP6135 Bug #1: Quantization Table checking - 16 bits
Segmentation fault (core dumped)
[av662490@net1547:~/CAP6135/PA2$ ./jpeg2bmp test-2.jpg out.bmp
CAP6135 Bug #2: More than 2 AC Huffman Tables
Segmentation fault (core dumped)
[av662490@net1547:~/CAP6135/PA2$ ./jpeg2bmp test-3.jpg out.bmp
CAP6135 Bug #3: Only support YCbCr image
Segmentation fault (core dumped)
[av662490@net1547:~/CAP6135/PA2$ ./jpeg2bmp test-4.jpg out.bmp
CAP6135 Bug #4: DC Huffman Table - more than 2
Segmentation fault (core dumped)
[av662490@net1547:~/CAP6135/PA2$ ./jpeg2bmp test-5.jpg out.bmp
CAP6135 Bug #5: too many Huffman tables
Segmentation fault (core dumped)
[av662490@net1547:~/CAP6135/PA2$ ./jpeg2bmp test-6.jpg out.bmp
CAP6135 Bug #6: Not a JPG file
Segmentation fault (core dumped)
[av662490@net1547:~/CAP6135/PA2$ ./jpeg2bmp test-7.jpg out.bmp
CAP6135 Bug #7: Huffman code not found
Segmentation fault (core dumped)
[av662490@net1547:~/CAP6135/PA2$ ./jpeg2bmp test-8.jpg out.bmp
CAP6135 Bug #8: Huffman decoding error
Segmentation fault (core dumped)
[av662490@net1547:~/CAP6135/PA2$ ./jpeg2bmp test-9.jpg out.bmp
CAP6135 Bug #9: More than 1KByte for Huffman Table
Segmentation fault (core dumped)
[av662490@net1547:~/CAP6135/PA2$ ./jpeg2bmp test-10.jpg out.bmp
CAP6135 Bug #10: Bogus JPEG format
Segmentation fault (core dumped)
[av662490@net1547:~/CAP6135/PA2$ █

```

--- Bugs Found ---

```

Bug 9 triggered 71 times
Bug 10 triggered 1134 times
Bug 3 triggered 2 times
Bug 1 triggered 84 times
Bug 6 triggered 112 times
Bug 8 triggered 16 times
Bug 7 triggered 4 times
Bug 5 triggered 2 times
Bug 2 triggered 3 times

```

Unique bugs found: 9

```

[av662490@net1547:~/CAP6135/PA2$ ls
cross.bmp  out.bmp      test-1.jpg  test-5.jpg  test-9.jpg
cross.jpg  PA2_mutations test-2.jpg  test-6.jpg  test.bmp
fuzzer.py  temp.bmp     test-3.jpg  test-7.jpg  test-temp.jpg
jpeg2bmp   test-10.jpg  test-4.jpg  test-8.jpg
[av662490@net1547:~/CAP6135/PA2$ █

```